Supabase Multi-Agent Project Guide

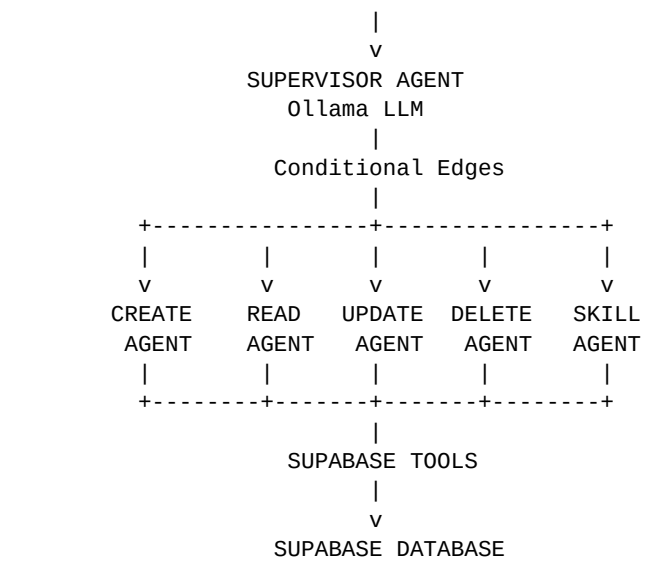
Below is a complete VS Code project for your use case: Ollama + Supabase + LangGraph + Supervisor Agent + mu

This implementation includes:

- Supervisor agent to route user requests.
- Create agent to insert records.
- Read agent to retrieve records.
- Update agent to modify records.
- Delete agent to delete records after explicit approval.
- Skill agent to load and execute instructions from .md skill files.
- LangGraph conditional edges for agent routing.
- Supabase connected to your existing database.
- Ollama as the local LLM.

I'll use a tasks table as an example. You can change the table name and columns to match your database.

1. Architecture



Skills are loaded from .md files by the agents.

2. Create the VS Code folder structure

Create a folder called supabase_multiagent.

```
supabase_multiagent/
|
+-- app.py
+-- config.py
+-- database.py
+-- agents.py
+-- graph.py
+-- requirements.txt
+-- .env
```

```
+-- .gitignore
|
+-- tools/
|   +-- __init__.py
|   +-- database_tools.py
|
+-- skills/
    +-- create_record.md
    +-- read_record.md
    +-- update_record.md
    +-- delete_record.md
    +-- database_policy.md
```

3. Install the dependencies

`requirements.txt`

Create this file:

langgraph
langchain
langchain-ollama
supabase
python-dotenv

Open the VS Code terminal:

python -m venv .venv

Activate the virtual environment.

Windows PowerShell:

.venv\Scripts\Activate.ps1

Linux/macOS:

source .venv/bin/activate

Install the dependencies:

```
pip install -r requirements.txt
```

```
Install Ollama from Ollama's official website (https://ollama.com/).
```

Download the model:

ollama pull llama3.1

Make sure Ollama is running:

ollama serve

If Ollama is already running as a background service, you do not need to start it again.

4. Configure Supabase

Create .env:

```
SUPABASE_URL=https://YOUR_PROJECT_ID.supabase.co
SUPABASE_KEY=YOUR_SUPABASE_SECRET_KEY

SUPABASE_TABLE=tasks

OLLAMA_MODEL=llama3.1
OLLAMA_BASE_URL=http://localhost:11434
```

Replace the placeholders with your actual credentials.

For this example, the existing Supabase table should contain:

Column Type Example

id Integer, primary key 1
title Text Build RAG application
description Text Create a Python RAG app
status Text pending

The id column should be automatically generated by your database.

If your existing table has different columns, modify the tool definitions in Section 7.

Security: Keep .env private. For production, use appropriate database permissions and Row Level Security. Never expose a service-role key to untrusted users.

.gitignore

```
.env
.venv/
__pycache__/
*.pyc
```

5. Configure the LLM

`config.py`

import os

from dotenv import load_dotenv
from langchain_ollama import ChatOllama

```
load_dotenv()

SUPABASE_URL = os.getenv("SUPABASE_URL")
SUPABASE_KEY = os.getenv("SUPABASE_KEY")

SUPABASE_TABLE = os.getenv(
    "SUPABASE_TABLE",
    "tasks"
)

OLLAMA_MODEL = os.getenv(
    "OLLAMA_MODEL",
    "llama3.1"
)

OLLAMA_BASE_URL = os.getenv(
    "OLLAMA_BASE_URL",
    "http://localhost:11434"
)
```

```
if not SUPABASE_URL or not SUPABASE_KEY:
    raise ValueError(
        "Configure SUPABASE_URL and SUPABASE_KEY in .env"
    )

llm = ChatOllama(
    model=OLLAMA_MODEL,
    base_url=OLLAMA_BASE_URL,
    temperature=0
)
```

This creates the LLM used by the supervisor and all specialized agents.

6. Connect to your existing Supabase database

`database.py`

from supabase import create_client

```
from config import (
    SUPABASE_URL,
    SUPABASE_KEY,
    SUPABASE_TABLE
)

supabase = create_client(
    SUPABASE_URL,
    SUPABASE_KEY
)

def get_table():
    return supabase.table(SUPABASE_TABLE)
```

This connects to the table specified in your .env file.

It does not create a new database.

7. Create the database tools

These are executable Python tools. They are different from Markdown skills and different from agents.

`tools/__init__.py`

Leave this file empty.

`tools/database_tools.py`

import json

from langchain_core.tools import tool

from database import get_table

```
@tool
def create_record(
    title: str,
    description: str,
    status: str = "pending"
) -> str:
    """Insert a task into Supabase."""
```

```
    if not title.strip():
        return "Error: title cannot be empty."

    try:
        result = (
            get_table()
            .insert({
                "title": title,
                "description": description,
                "status": status
            })
            .execute()
        )

        return json.dumps({
            "success": True,
            "operation": "create",
            "records": result.data
        }, default=str)

    except Exception as exc:
        return f"Create failed: {exc}"

@tool
def read_records(
    record_id: int = 0
) -> str:
    """Read tasks. Use zero to retrieve multiple records."""

    try:
        query = get_table().select("*")

        if record_id > 0:
            query = query.eq("id", record_id)

        result = query.limit(100).execute()

        return json.dumps({
            "success": True,
            "operation": "read",
            "records": result.data
        }, default=str)

    except Exception as exc:
        return f"Read failed: {exc}"

@tool
def update_record(
    record_id: int,
    title: str = "",
    description: str = "",
    status: str = ""
) -> str:
    """Update selected fields of a task identified by its ID."""

    if record_id <= 0:
        return "Error: a valid record ID is required."

    updates = {}

    if title:
        updates["title"] = title

    if description:
```

```
        updates["description"] = description

    if status:
        updates["status"] = status

    if not updates:
        return "Error: no update fields supplied."

    try:
        result = (
            get_table()
            .update(updates)
            .eq("id", record_id)
            .select("*")
            .execute()
        )

        return json.dumps({
            "success": True,
            "operation": "update",
            "records": result.data
        }, default=str)

    except Exception as exc:
        return f"Update failed: {exc}"

@tool
def delete_record(
    record_id: int,
    approved: bool = False
) -> str:
    """
    Delete a task by ID.

    approved must be true, but this argument alone
    is not a secure human-approval mechanism.
    """

    if record_id <= 0:
        return "Error: a valid record ID is required."

    if not approved:
        return (
            f"Deletion of task {record_id} was not executed. "
            "Explicit approval is required."
        )

    try:
        result = (
            get_table()
            .delete()
            .eq("id", record_id)
            .select("*")
            .execute()
        )

        return json.dumps({
            "success": True,
            "operation": "delete",
            "records": result.data
        }, default=str)

    except Exception as exc:
        return f"Delete failed: {exc}"
```

```
CRUD_TOOLS = [
    create_record,
    read_records,
    update_record,
    delete_record
]
```

The tools return the results reported by Supabase. An empty records list can mean that the specified ID did not match any accessible record.

```
Production note: A real deletion approval must be enforced by the application or workflow, not merely by an
```

8. Create the Markdown skills

These files contain the instructions that the agents load at runtime.

```
skills/create_record.md
```

```
# Create Record Skill

## Objective
Create a new record in the configured Supabase table.

## Procedure
1. Understand the user's request.
2. Identify the title, description, and status.
3. Ask for missing required information.
4. Validate the values.
5. Call create_record.
6. Inspect the returned result.
7. Report the actual database outcome.

## Rules
- Do not invent field values.
- Do not claim success without tool confirmation.
- Do not execute arbitrary SQL.
- Use the configured table only.
```

```
skills/read_record.md
```

```
# Read Record Skill

## Objective
Retrieve records from Supabase.

## Procedure
1. Determine whether the user wants one record or multiple records.
2. Obtain the record ID when required.
3. Call read_records.
4. Inspect the returned records.
5. Present the actual data clearly.

## Rules
- Never invent records.
- Do not claim access to records that were not returned.
- Respect database permissions.
```

```
skills/update_record.md
```

```
# Update Record Skill

## Objective
```

```
Update an existing record.

## Procedure
1. Identify the record ID.
2. Determine which fields should change.
3. Ask for missing information.
4. Call update_record.
5. Inspect the returned records.
6. Report the actual outcome.

## Rules
- Always specify the record ID.
- Update only the requested fields.
- Do not claim success if the update fails.
```

skills/delete_record.md

```
# Delete Record Skill

## Objective
Delete a record only after explicit user approval.

## Procedure
1. Identify the record ID.
2. Explain which record will be deleted.
3. Obtain approval through the application workflow.
4. Execute the deletion only after approval.
5. Inspect the result.
6. Report the outcome.

## Rules
- Never delete an ambiguously identified record.
- Never treat an LLM-generated approval as user approval.
- The application must enforce approval.
```

skills/database_policy.md

```
# Database Policy Skill

## Rules
1. Use only the configured Supabase table.
2. Never execute arbitrary SQL.
3. Do not invent database results.
4. Do not expose credentials.
5. Use record IDs for updates and deletions.
6. Verify the result of every mutation.
7. Ask questions when required information is missing.
8. Respect the application's authorization rules.
```

9. Create the agents

This is where we create five actual agents, not five tools.

The agents share the Ollama model but have separate instructions and responsibilities.

agents.py

```
from pathlib import Path

from langchain.agents import create_agent

from config import llm
```

```
from tools.database_tools import (
    create_record,
    read_records,
    update_record,
    delete_record
)

# ----------------------------
# Markdown skill loader
# ----------------------------

SKILLS_DIR = Path(__file__).parent / "skills"

def load_skill(skill_name: str) -> str:
    path = SKILLS_DIR / f"{skill_name}.md"

    if not path.is_file():
        raise FileNotFoundError(
            f"Skill not found: {skill_name}"
        )

    return path.read_text(encoding="utf-8")

DATABASE_POLICY = load_skill("database_policy")

def build_prompt(skill_name: str) -> str:
    return (
        load_skill(skill_name)
        + "\n\n"
        + DATABASE_POLICY
    )

# ----------------------------
# Agent 1: Create
# ----------------------------

create_agent = create_agent(
    model=llm,
    tools=[create_record],
    system_prompt=build_prompt("create_record")
)

# ----------------------------
# Agent 2: Read
# ----------------------------

read_agent = create_agent(
    model=llm,
    tools=[read_records],
    system_prompt=build_prompt("read_record")
)

# ----------------------------
# Agent 3: Update
# ----------------------------

update_agent = create_agent(
    model=llm,
    tools=[update_record],
    system_prompt=build_prompt("update_record")
)

# ----------------------------
# Agent 4: Delete
```

```
# ----------------------------

delete_agent = create_agent(
    model=llm,
    tools=[delete_record],
    system_prompt=build_prompt("delete_record")
)

# ----------------------------
# Agent 5: Skill
# ----------------------------

skill_agent = create_agent(
    model=llm,
    tools=[],
    system_prompt=(
        "You are a skill assistant. "
        "Explain and apply the database procedures "
        "described in the supplied Markdown skills. "
        "Do not access the database or claim to execute "
        "operations. You can answer questions about "
        "the procedures and policies.\n\n"
        + DATABASE_POLICY
    )
)
```

Important Python naming detail: The create_agent function imported from LangChain is also used above as a variable name. That shadows the imported function and causes an error when the next agent is created.

To avoid that, use an alias for the imported function. Replace the import:

from langchain.agents import create_agent

with:

from langchain.agents import create_agent as make_agent

`Then change each initialization to use make_agent(...), naming the instances create_agent, read_agent, updat`

The corrected initialization pattern is:

```
create_agent = make_agent(
    model=llm,
    tools=[create_record],
    system_prompt=build_prompt("create_record")
)
```

Apply that pattern to all five agents.

10. Build the LangGraph supervisor

The supervisor chooses which specialist agent to execute.

`graph.py`

from typing import TypedDict, Literal

from langchain_core.messages import HumanMessage
from langgraph.graph import StateGraph, START, END

from config import llm

```
from agents import (
    create_agent,
    read_agent,
    update_agent,
    delete_agent,
    skill_agent
)

class AgentState(TypedDict):
    user_request: str
    route: str
    result: str

# ---------------------------------
# Supervisor
# ---------------------------------

def supervisor_node(state: AgentState):
    prompt = f"""
You are the supervisor of a Supabase multi-agent system.
```

Select exactly one route from:

CREATE
READ
UPDATE
DELETE
SKILL
FINISH

Routing rules:

CREATE:
Insert a new record.

READ:
Retrieve one or more records.

UPDATE:
Modify an existing record.

DELETE:
Delete a record.

SKILL:
Explain database skills, Markdown instructions,
or database policies without performing database operations.

FINISH:
Greetings or general questions.

```
User request:
{state["user_request"]}
```

Return only the route name.
"""

```
    response = llm.invoke(prompt)
    route = response.content.strip().upper()

    valid_routes = {
```

```
        "CREATE",
        "READ",
        "UPDATE",
        "DELETE",
        "SKILL",
        "FINISH"
    }

    if route not in valid_routes:
        route = "FINISH"

    return {"route": route}

# ---------------------------------
# Helper for invoking an agent
# ---------------------------------

def invoke_agent(agent, state: AgentState):
    response = agent.invoke({
        "messages": [
            HumanMessage(content=state["user_request"])
        ]
    })

    return {
        "result": str(response["messages"][-1].content)
    }

# ---------------------------------
# Specialist agent nodes
# ---------------------------------

def create_node(state):
    return invoke_agent(create_agent, state)

def read_node(state):
    return invoke_agent(read_agent, state)

def update_node(state):
    return invoke_agent(update_agent, state)

def delete_node(state):
    return invoke_agent(delete_agent, state)

def skill_node(state):
    return invoke_agent(skill_agent, state)

def finish_node(state):
    response = llm.invoke(
        "Answer the following general question. "
        "Do not claim database operations were performed.\n"
        + state["user_request"]
    )

    return {"result": str(response.content)}

# ---------------------------------
# Conditional routing
# ---------------------------------

def route_request(
    state: AgentState
) -> Literal[
    "create",
    "read",
```

```
    "update",
    "delete",
    "skill",
    "finish"
]:
    routes = {
        "CREATE": "create",
        "READ": "read",
        "UPDATE": "update",
        "DELETE": "delete",
        "SKILL": "skill",
        "FINISH": "finish"
    }

    return routes[state["route"]]

# --------------------------------
# Build graph
# --------------------------------

workflow = StateGraph(AgentState)

workflow.add_node("supervisor", supervisor_node)
workflow.add_node("create", create_node)
workflow.add_node("read", read_node)
workflow.add_node("update", update_node)
workflow.add_node("delete", delete_node)
workflow.add_node("skill", skill_node)
workflow.add_node("finish", finish_node)

workflow.add_edge(START, "supervisor")

workflow.add_conditional_edges(
    "supervisor",
    route_request,
    {
        "create": "create",
        "read": "read",
        "update": "update",
        "delete": "delete",
        "skill": "skill",
        "finish": "finish"
    }
)

for node in [
    "create",
    "read",
    "update",
    "delete",
    "skill",
    "finish"
]:
    workflow.add_edge(node, END)

app = workflow.compile()
```

What makes this multi-agent?

The graph contains a supervisor and six execution nodes. Five nodes invoke distinct agents with different tool permissions and Markdown instructions; the sixth handles general questions.

The supervisor uses LangGraph conditional edges to choose the appropriate agent.

For example:

```
User: Update task 10 status to completed
                 |
                 v
          Supervisor Agent
                 |
          route = UPDATE
                 |
                 v
            Update Agent
                 |
          update_record tool
                 |
                 v
           Supabase UPDATE
                 |
                 v
             Final Result
```

11. Create the application entry point

`app.py`

from graph import app

```
def main():
    print("=" * 60)
    print("SUPABASE MULTI-AGENT SYSTEM")
    print("LLM: Ollama")
    print("Framework: LangGraph")
    print("Skills: Markdown files")
    print("=" * 60)

    print("\nAvailable requests:")
    print("1. Create a task")
    print("2. Show all tasks")
    print("3. Update task 1 status to completed")
    print("4. Delete task 1")
    print("5. Explain the create skill")
    print("\nType 'exit' to quit.\n")

    while True:
        request = input("You: ").strip()

        if request.lower() in {"exit", "quit"}:
            print("Goodbye!")
            break

        if not request:
            continue

        try:
            result = app.invoke({
                "user_request": request,
                "route": "",
                "result": ""
            })

            print("\nSelected agent:", result["route"])
            print("Assistant:", result["result"])
```

```
            print()

        except Exception as exc:
            print(f"\nApplication error: {exc}\n")

if __name__ == "__main__":
    main()
```

12. Run the project in VS Code

Start Ollama first, then run:

```
python app.py
```

Try these requests.

Create a record

Create a task titled Build RAG Application
with description Implement Python RAG
and status pending.

Expected routing:

```
SUPERVISOR -> CREATE AGENT -> CREATE TOOL -> SUPABASE
```

Read records

Show all tasks.

Expected routing:

```
SUPERVISOR -> READ AGENT -> READ TOOL -> SUPABASE
```

Update a record

Update task 10 status to completed.

Expected routing:

```
SUPERVISOR -> UPDATE AGENT -> UPDATE TOOL -> SUPABASE
```

Explain a skill

Explain the procedure used by the create record skill.

Expected routing:

```
SUPERVISOR -> SKILL AGENT -> RESPONSE
```

The skill agent explains procedures but does not perform database operations.

13. Important limitation: delete approval

The current example's delete agent is not yet a secure human-in-the-loop implementation. The